1 You can do it!

This tutorial is meant to be interactive... that is *you* should be reading, typing, thinking, and asking questions. There are no source files to share, the point is for you to think, enter the code, and see the result. And then, to vary the code and see what changes. This is a more effective strategy than just *reading* in code others have written and running it. Indeed, the ideal way to use this book is not to copy and paste code. Instead, we recommend typing in the code for yourself in nearly all circumstances. The act of typing will help you understand how to implement basic commands, debug and refine your programs, and structure your approach to building computational models of living systems. A few notes:

- If you are experienced in R, then please work on challenge problems identified in this tutorial.
- Please do talk to others via chat if you're stuck, but write your own code!
- You can do it!

The philosophy underlying this hands-on approach is explained in the Preface. It is now in your hands to realize.

2 Getting Started

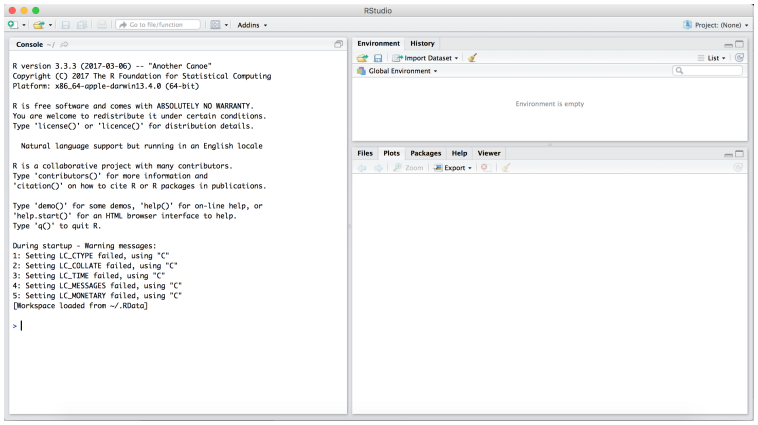
R is an open-source programming language for statistical computing and graphics. R is well known for its rich package ecosystem, as well as the ease with which one can produce good-quality plots and graphics. We will be using R in this laboratory guide to simulate and model the dynamics of living systems from molecules, cells, organisms, to ecosystems. R is particularly good at:

- Data manipulation
- Statistics
- Visualization
- Interfacing with other languages

and is okay at:

- Data & memory handling
- Speed (as compared to Python and Matlab)

There are a variety of Integrated Development Environments (IDEs) for R. For this course we will use RStudio. So, let's get started! This chapter will help you gain practical experience using R. When you open R, you should see a set of windows that looks like this:



The key elements are the Command Window, File Editor, Environment Window, and Help Window. The command window is where you enter commands, and you should see a prompt that looks like this:

```
>
```

You can do basic math at this prompt (in order to run the cell press Enter), for example:

```
> 3+4
[1] 7
```

and

```
> exp(1)
[1] 2.718282
```

Alternatively, you can write your commands in the script file (top left) and press ctrl + Enter to run it in the console.

One of R's greatest strength is the diverse library of packages available to it. There are an incredible amount of packages that are useful for everything from 3D design to Genomics. Making use of this diverse set of packages requires that they be loaded into the current workspace. Installing and importing them is simple. Remember, that in every new session you must load the packages you wish to use. For example, to install the "fitdistrplus" package you can type in the console:

```
[obeytabs,tabsize=4]
    > install.packages("fitdistrplus")
```

And then load it using:

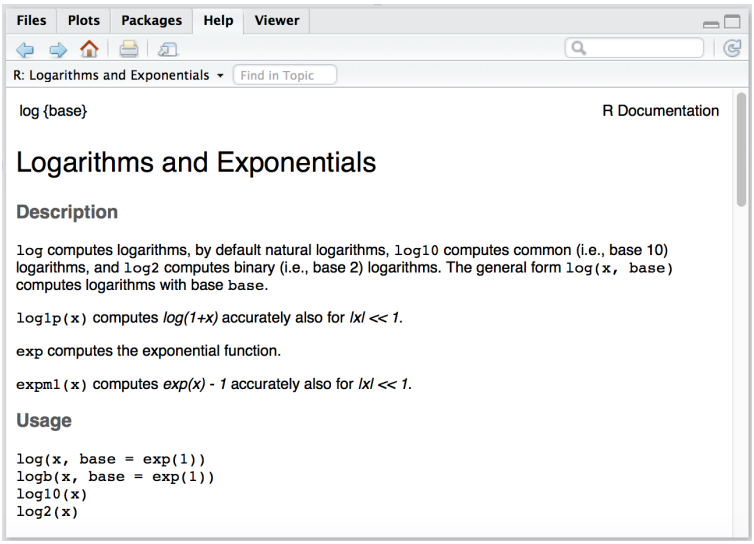
```
> library("fitdistrplus")
```

Once a package is loaded, it is made available for every file opened in RStudio, so be careful when jumping between files. It is good practice to have import statements in the first few lines of every file.

R has several functions that are built in (and you can use it like a calculator). For example, to learn more about the exponential function:

```
>?exp
```

A help page should pop to your right. It looks like this:



Many functions have names that you expect (how do you think you should calculate cosine or sine of a value, for example)? Try it out!

If you don't know the name of the function you can use ?? followed by a key word.

```
> ??trignonometry
```

R is not just a calculator. It is also a programming language that can store values in memory. For example, the command

```
> x = 3
> x
[1] 3
```

tells R that the variable x has the value 3 and now every time you use "x", R will substitute the value 3, for example:

```
> y = x + 1
> print(y)
[1] 4
```

It is very important to realize the "=" sign does not mean that R checks to see if the two sides are equal to each other. Instead, R interprets the = sign to assign the value on the right to the variable on the left. If you want to check the truth of a particular statement – is x equal to 3, or alternatively, is y equal to 3 – then you would type:

```
> x==3
[1] TRUE

> y==3
[1] FALSE
```

The double "==" sign tells R to logically compare what is on the left with that on the right and return 1 if true and 0 if false. Note that if you want R to report back the answer/value you can either call the variable or use a print() function.

R can also handle arrays of values (for example a vector or a matrix). The simplest way is to use the colon, which defines a sequential vector, for example

```
> v=1:5
> v
[1] 1 2 3 4 5
```

you can also modify the increments by adding a step parameter at the end of the function seq()

```
[obeytabs,tabsize=4]
> w = seq(1,9,2)
> print(w)
[1] 1 3 5 7 9
```

Any entry can be examined using the brackets

```
> w[3]
[1] 5
```

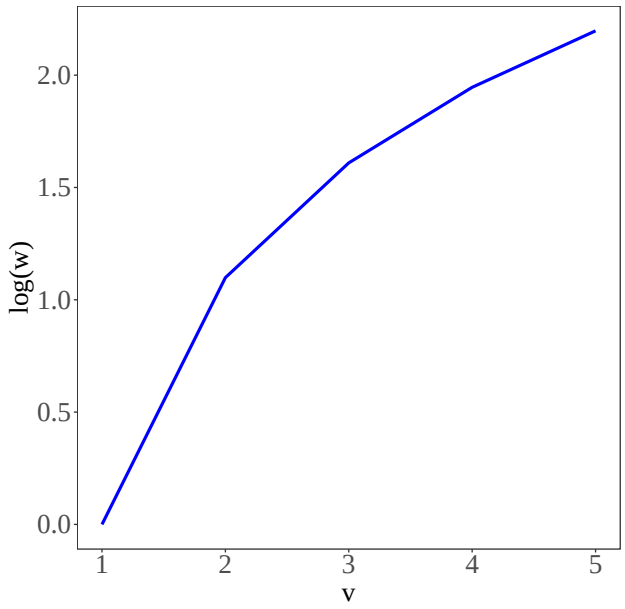
and basic math can be performed automatically on vectors (and matrices), for example

```
> log(w)
[1] 0.000000 1.098612 1.609438 1.945910 2.197225
```

R can also plot graphs, surfaces, and more. To create a simple plot, use the plot command

```
> require(ggplot2)
> ggplot() +
    geom_line(aes(v,log(w)))
```

which leads to:



3 Building "Programs" from "Scripts" and "Functions"

Once you have a lot of commands, it will get exhausting typing them again and again (especially when you make mistakes). Instead, you will want to use a "script". A script is a list of commands in a file that you can execute directly from the command window. To create a script go to the File menu and select New File > R Script. Now type in a few commands, such as:

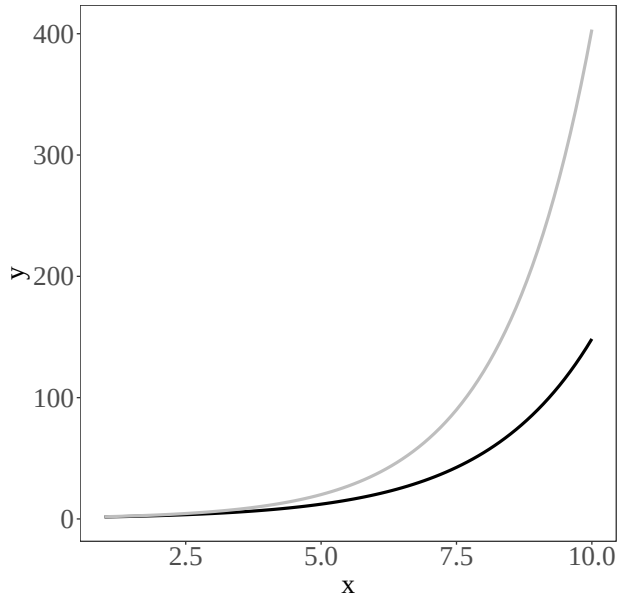
```
#my_first_file.R

# Create some vectors
x = seq(1,10,0.1)
y1 = exp(0.5*x)
y2 = exp(0.6*x)

#plot the vectors
ggplot() +
  geom_line(aes(x,y1),color="black") + #use a black line
  geom_line(aes(x,y2),color="grey") + # use a grey line
  #add labels
  xlab("x") +
  ylab("y")

#save the image to a file
dev.copy(png,"my_first.png")
dev.off()
```

Save this file as my_first_file.R in the same folder you're currently working in. Now click on the "Source" button at the top bar of Rstudio and it should execute the commands in the script to yield the figure:



The problem with this script is that changing the arguments in the exponential functions requires editing the script and then re-running the code. It would be more convenient to designate a variable change from the command window and have the code automatically update its output. The problem is that a script cannot return a variable or accept a variable as input. To do so requires a "function". Functions are program files that can be called from the Command window, can accept inputs, and return outputs. To start one, open a new file and type:

```
# My first function
logGrowth = function(t,N,...){
    #function dNdt = logGrowth(t,N)
    #logGrowth gives the growth rate of a population of size N at time t
    #usage: dNdt = logGrowth(t,N)
    r = 0.5
    K = 100
    dNdt = r*N*(1-N/K)
    return(list(dNdt))
}
```

Now the new function can be accessed just like one of R's built-in functions, for example, type the following code into a new file named "Lab1s2.R". Note that to use code from another file you need to source the file

```
source("Lab1_Functions.R")
vN = 0:110
ggplot() +
  geom_line(aes(vN,logGrowth(0,vN)[[1]])) +
  xlab("N") +
  ylab("dN/dt")
```

This gives an upside down parabola, denoting that growth rate is positive between 0 and 100 and negative when N is greater than 100. Note that the argument `t` is not used in the `logGrowth` function. Not all inputs have to be used. We will update this function later to both accept and utilize all inputs.

4 Getting Started with Core Techniques

4.1 Create loops

There are many loops that enable the repetition of a fixed set of commands. The two central loops examined here are `for` and `while` loops. Both loops start with a keyword such as `for` or `while` and repeat the code within curly brackets. The `for` loop allows us to repeat certain commands many times with a "counter" variable. Here is one example:

```
> for(j in 1:4){ # counting
+   j+2 # the statement you want to repeat
+ }
```

A counter variable can be incremented by any real number, e.g.,:

```
> for(j in seq(1,0,-0.1)){ # counting
+   print(1-j) # the statement you want to repeat
+ }
```

For loops can also span an arbitrary set of values:

```
> for (j in c(1,3,5,7)){
+   print(1-j)
+ }
```

Challenge Problem: Exercise on Matrices

Define a random matrix A of size 3-by-3. Use a double for loop to calculate the square of the entries in A and store the values in another matrix B. (Hint: type ?runif if you don't know how to define a random matrix)

Solution to Challenge Problem: Exercise on Matrices A random 3x3 matrix and its element by element square can be generated as follows.

```
# Define a random matrix and an empty matrix
A=matrix(runif(3*3),nrow=3,ncol=3)
B=matrix(NA,nrow=3,ncol=3)
# Square each element
for(i in 1:3){
    for(j in 1:3){
        B[i,j]=A[i,j]^2
    }
}
```

A `while` loop is the most robust of loops; formally speaking it is a universal loop such that all loops can be built using it. That is a topic for some other class – for our purposes, both `while` and `for` loops may be appropriate depending on the circumstance. The `while` loop repeats a sequence of commands as long as some condition is met. For example, given a number n, the following code will return the smallest non-negative integer a such that $2^a \geq n$.

```
smallexp = function(n){
    a = 0
    while(2**a < n){
    a = a+1} # statement to execute if the condition
    return(a)
    }
```

such that this command can be invoked from the R interface:

```
> a = smallexp(4)
> print(a)
[1] 2
```

Note that in the above example we used the conditional statement, $2^a < n$ to decide whether the statement within the while loop should be repeated. Such conditional statements are also used in `if` statements. The relational operator used here is "<", which means "less than". Other relational operators that are available in R include:

```
>  greater than
<= less than or equal
>= greater than or equal
== equal
!= not equal
```

Simple conditional statements can be combined by logical operators

```
(&, |, !)
```

into compound expressions such as the following:

```
(y > 1) & (x == 6)
```

4.2 Make decisions

Now, suppose you want your code to make a decision. In many circumstances, a series of `if` statements will suffice. The general form in R is as follows:

```
if(expression1){
  statements1
} else if(expression2){
  statements2
} else {
  statements3
}
```

Challenge Problem: Recursive Factorial

Finish the following pseudo-code that gives the factorial of a positive number n, using the recursive formula $n! = n(n-1)...1$

```
factorial_recur <- function(n){
    if ( # ){
     N = 1
 } else{
     N = #
 }
 return(N)
}
```

Solution to Challenge Problem: Recursive Factorial
The solution to this recursive problem involves thinking about what happens in a sequence. For example, if $3! = 3 \times 2 \times 1$ that is equivalent to $3! = 3 \times 2! = 3 \times 2 \times 1! = 3 \times 2 \times 1$. In essence, by breaking the big problem into little problems of the same kind it is possible to get to the point where the answer is known and then reconstruct the solution – in the formula above it would be moving from left to right and back again. Here is one such solution. Note that one would have to add error correction to make sure there are not issues if the user calls the functions with a real number or a non-positive integer.

```
factorial_recur <- function(n){
    if (n==1){
     N = 1
 } else{
     N = n*factorial_recur(n-1)
 }
 return(N)
}
```

4.3 Go fast, i.e., "vectorize"

Remember: `for` loops are SLOW in R. One way to make your R run faster is to `vectorize` the algorithm you use in the code. Vectorization can be done by converting `for` and `while` loops to equivalent vector or matrix operations. A simple example would be the following System.time is a function that captures the run time of your command.

```
> x<-0; y<-c()
> system.time(
+   for(i in 1:1000){
+     y <- c(y,log10(x))
+     x <- x+0.01
+   }
+ )

> system.time(
+   {
+   x <- seq(0,10,0.01);
+   y <- log10(x)
+   }
+ )
```

You should find that the second command set is faster, even if it returns exactly the same output. However, for more complicated code, vectorization is not always so obvious. Some of the most commonly used functions for vectorizing can be found in the Help browser of R.

4.4 Find values you want to know about

Given an array X, the command ¿ which(X!=0) returns the indices of all nonzero elements of X. You can also use a logical expression to define X. For example,

```
> which(X>2)
```

returns the indices corresponding to the entries of X that are greater than 2. Notice that X could also be a matrix. In this case, the function returns the linear indices as if the matrix was stored as a single column of elements (try it!). The 'elementwise' logical operators (& and |) can be used to locate the entries that satisfy more than one logical expressions. These commands will be important in solving the next challenge problem.

Challenge Problem: Finding Elements in Matrices

Build a 5-by-5 random matrix A using the command

```
> A <- matrix(runif(5*5),5,5)
```

Find the indices of entries whose values are smaller than 1/4 and bigger than 1/6. Check the answer with your (virtual) neighbors or study partners.

Solutions to Challenge Problem: Finding Elements in Matrices
Given that `matrix(runif(5*5),5,5)` generates a 5×5 matrix, finding linear indices can be done in a few ways. One way is to use the `which` command, as follows:

```
ind = which((A>(1/6)) & (A<(1/4)))
```

Note that this answer can be empty if there are no elements that satisfy the condition. Even though the matrix is square the indices will always be between 1 and 25 (inclusive, because there are 25 elements). By default, R indexes matrices column by column so that the fourth element is in the fourth row and first column and the eighth element is in the third row and third column.

4.5 Save and load data

Saving and loading objects (variables,arrays, or other forms of data) can be done in a multitude of ways in R, however; for this course we will use the native R format.

```
> array <- c(10,20,30,40)
> save(array, file = "array.RData")
```

and R will save the specified array in the file name. When you want to load all the variables from the file specified by filename, just type

```
[obeytabs,tabsize=4]
    > load("array.RData")
```

Please keep in mind that these file formats are binary and proprietary, i.e., not human-readable. If you want to save a particular file format use the help to save variables in standard, comma-separated value file format. There are also alternative ways to load and print data.

5 Numerically Integrating Differential Equations

In this class we can use R to numerically solve differential equations, like the logistic growth equation, even when such solutions are not available analytically. The most-used R program which does the integration is called `ode` from the deSolve package. Here is a script that integrates the logGrowth function. We will return to this multiple times in the course.

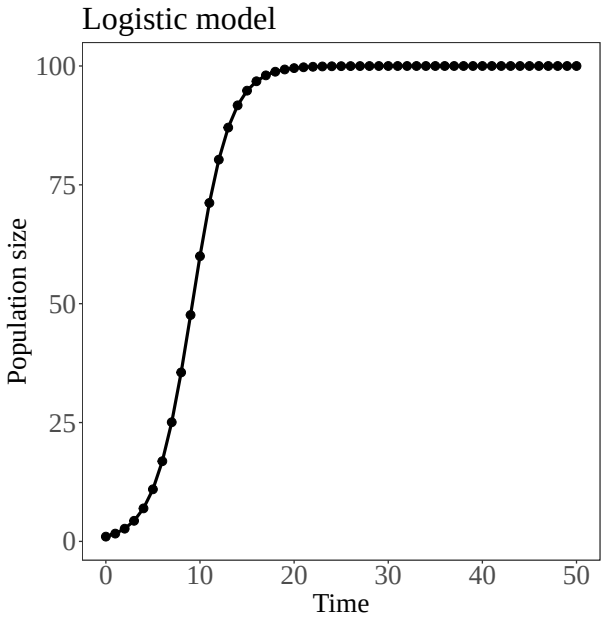
```
# Numerical solution of the logistic equation
require(deSolve)
require(ggplot2)
source("Lab1_Functions.R")
t0 =0  # Initial time
tf =50 # Final time
N0 =1  # Initial population size
y = ode(N0,t0:tf,logGrowth)#Numerically integrate
T=y[,1]
Nint=y[,2]
# Actual solution
```

```
r =0.5
K =100
Nact =(N0*exp(r*T))/(1+N0*(exp(r*T)-1)/K) #Actual solution
# Plot results
ggplot() +
  geom_line(aes(T,Nint)) +
  geom_point(aes(T,Nact)) +
  xlab("Time") +
  ylab("Population size") +
  ggtitle("Logistic model")
dev.copy(pdf,"logPlot.pdf")
dev.off()
```

Save this script as intLog.R and run it

```
> source("intLog.R")
```

which yields the plot



Here are some important points to keep in mind:

- R solves ordinary differential equations of the form

$$\frac{d\vec{y}}{dt} = \vec{f}(\vec{y})$$

where $\vec{f}(\vec{y})$ is a vector of functions. The default solver used in these computational laboratories is **ode**.

$$\text{YOUT} = \text{ode(Y0,TSPAN,ODEFUN)}$$

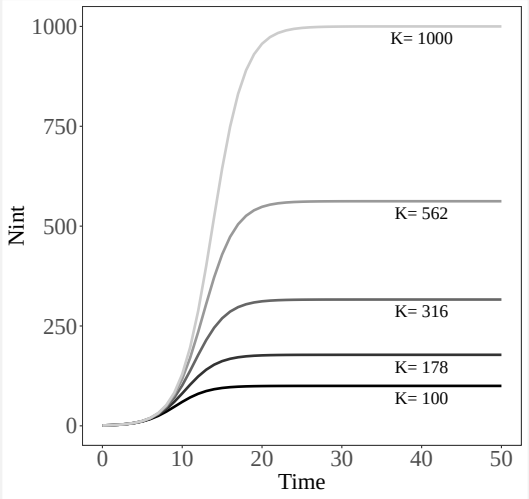
where

1. YOUT $\rightarrow$ value of variable
2. ODEFUN $\rightarrow$ name of function containing the dynamics
3. TSPAN $\rightarrow$ time limits for integration
4. Y0 $\rightarrow$ initial conditions

and further documentation on deSolve's `ode` may be found online. The key idea is that R has a built-in function that numerically integrates a differential equation that must be specified – by you. It integrates the system of equations over a range of time given initial conditions. Give this a try, albeit by going farther: accepting additional parameters as input to the dynamical system.

Challenge Problem: Variables and Differential Equations

Read the `odeint` documentation and modify your logistic growth model to accept the parameters r and K. Setting $r = 0.5$, vary the value of K over 1 order of magnitude and compare the results using a plot command. If your code works, the resulting figure should look something like this:



Solutions to Challenge Problem: Variables and Differential Equations

Although some variables may be fixed in a given system of equations, there are many circumstances where it is necessary to probe the response of a system to variation in such conditions. In this case, it is possible to pass variables to the `ode45` command, which will, in turn, pass these variables on the function specified to actually compute the rate of change in the system of equations. It's best seen by example.

First, here is a modified code for `intLog.m` that generates a structure of parameters called `pars` and sets up a loop to change the value of K:

```
logGrowth_withpars = function(t,N,pars){
  # function dNdt=logGrowth_withpars(t,N,pars)
  #
  # logGrowth gives the growth rate of a population of size N at time t
  # Usage:dNdt=logGrowth(t,N,pars)
  # pars is a structure that contains parameters
  r = pars$r
  K = pars$K
  dNdt =r*N*(1-N/K)
  return(list(dNdt))
}
```

Next, here is a script that calls this new function multiple times, each with a new value of K and then plots the dynamics on the same axes.

```
source("Lab1_Functions.R")
require(ggplot2)
require(ramify)
# Parameters
t0 =0  # Initial time
tf =50 # Final time
N0 =1  # Initial population size
pars=list()
pars$r=0.5
Krange =logspace(2,3,5)  # Generate 5 logarithmically spaced values
colors =paste("gray",seq(0,80,length.out = length(Krange)),sep="") #Generate scale of grays

g=ggplot()
for (i in 1:length(Krange)){
  pars$K=Krange[i]  # Set the value of K
  y = ode(N0, t0:tf, logGrowth_withpars,pars) #Numerically integrate
  y=data.frame(y)
  colnames(y)=c("Time","Nint")
  g = g +
    geom_line(data=y,aes(Time,Nint),color=colors[i]) +
    geom_text(data=data.frame(x=40,y=pars$K-30),
    aes(x,y),
    label=paste("K=",pars$K),
    color="black")
}
print(g)
```

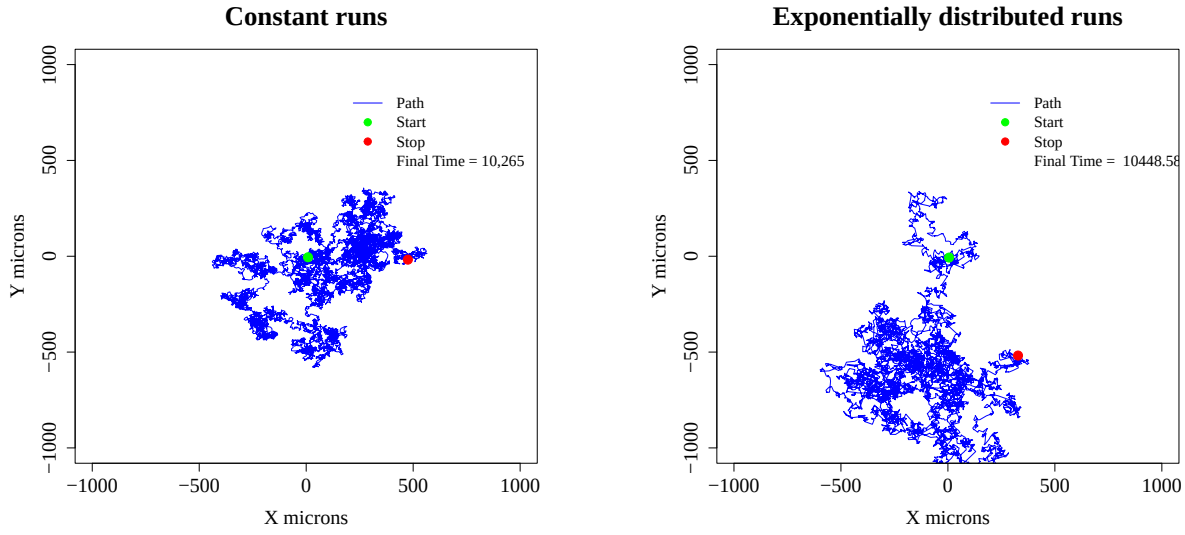
6 Advanced - Individual-Based Simulations

As a final challenge for those more experienced in R, try and develop a simulation of "diffusion" in which a bacteria swims at a speed of 10 μm/s with each "run" lasting 1 second. In this case the direction of the next run is random. How long will it take, on average, for the bacteria to travel 1mm away from its source? As you may recall, the average time for a diffusing particle or organism to travel a squared distance x^2 on average should be:

$$T = \frac{x^2}{D} \tag{1}$$

It is true that any particular bacteria may travel much farther. What do we mean by *average* here? Indeed, the average distance traveled by bacteria is 0. That is the bacteria is just as likely to go to in any particular direction. But if one squares the distance from the origin, you will find that the average squared distance increases linearly with time.

If you develop code, can you visualize the results? If you change the length in equation, how does T change? Can you confirm the scaling and estimate D? What happens if the durations of each run is exponentially distributed, so that runs last on average 1 second, but can vary in duration? Did the answer with respect to travel time to reach 1mm change in a quantitative or qualitative way? We will get much further into these ideas later in the class. Here are examples of two runs, one with constant duration and one with exponentially distributed durations:



7 Take-home messages for the computational laboratories ahead

- Building computational models of living systems requires a willingness to code, tinker, try, and try again.
- Expanding your computational toolset will take time.
- Mistakes are part of the learning process and indeed, will help reinforce common pitfalls.
- With time, your objective should be able to code for the most part without having to open a help forum or to try and search for the answer in your favorite search engine.
- The basic skills presented here are not comprehensive, instead they are a gateway to further exploration.
- The modules that follow build on a few of the methods in this laboratory, with a focus on direct integration of computational modeling as an integral part of the quantitative biosciences approach to science.

- You can do it! Really.